

AI Guild Hackathon Report

Team Name: 405 Found

Aadi, Rahul, Utkarsh

September 16, 2024

1 Approach

1.1 Pre-processing

1. **Data Distribution Analysis:** We analyzed the distribution of emotions in the dataset and found significant class imbalance. Certain emotions had considerably fewer images than others, which could potentially lead to the model being biased towards the more prevalent classes.
2. **Pixel Data Conversion:** The raw pixel input data was converted into NumPy arrays of dimension 48×48 . This step was essential to shape the data into a format compatible with deep learning frameworks.
3. **Grayscale to RGB Conversion:** As all the images were in grayscale (single channel), we converted them to RGB (three channels) since many pre-trained models are designed to work with RGB images. This allowed us to perform transfer learning by using models pre-trained on the ImageNet dataset.
4. **Label Encoding:** We applied one-hot encoding to the emotion labels to convert categorical data into a numerical format. This step ensured that the model treated each emotion as a distinct class during training.
5. **Handling Class Imbalance using SMOTE:** Implemented SMOTE (Synthetic Minority Over-sampling Technique) to oversample minority classes. However, we applied SMOTE only to specific classes (excluding 'Happy') to avoid generating too much synthetic data, which could lead to overfitting. This step helped in creating a more balanced training set, improving the model's ability to learn features of underrepresented emotions.
6. **Data Augmentation:** To further enhance the dataset, we applied various augmentation techniques such as rotation, flipping, zooming, shifting, and brightness adjustment. Data augmentation increases dataset diversity, enabling the model to generalize better on unseen data. These augmentations simulate real-world variations, helping the model to become more robust to input changes.
7. **Class Weights Assignment:** Assigned class weights inversely proportional to the number of samples in each class. This weighting penalizes misclassification of minority classes more than majority classes during training, guiding the model to learn the features of less frequent emotions effectively.

1.2 Model Training

- **Model Selection:** We experimented with several models, including:
 - **Convolutional Neural Networks (CNNs):** VGG19, MobileNetV2, ConvNext, EfficientNetB0, EfficientNetV2B3, ResNet50, DenseNet.
 - **Transformers:** ViT (Vision Transformer), PiT (Pyramid Vision Transformer), SWIN Transformer.
 - **Ensemble Approaches:** Implemented weighted ensembles to combine the strengths of multiple models. Examples of ensembles we tried include:
 - * (ConvNext, EfficientNetV2B3)
 - * (VGG19, EfficientNetB0)

- * (EfficientNetB0, ResNet50, DenseNet)
- * (ViT, EfficientNetB0)

The ensemble approach aims to combine the predictive capabilities of various architectures to enhance overall accuracy, especially in handling class imbalance.

- **Optimization and Loss Function:** Initially used the Adam optimizer with categorical cross-entropy loss. We later switched to **Focal Loss** to address the class imbalance more effectively. Focal loss focuses on hard-to-classify samples, ensuring that the model learns difficult cases by down-weighting easy examples.
- **Callbacks for Better Training:** Utilized callbacks such as **Early Stopping** to halt training when validation performance stagnated, preventing overfitting. We also employed **ReduceLROnPlateau** to lower the learning rate when validation accuracy plateaued, allowing the model to converge to a better solution.

1.3 Post-processing

1. **Model Evaluation:** After training, we evaluated the model on the validation set using metrics such as accuracy, precision, recall, and F1-score. We studied the classification report to analyze the model's performance on each emotion. The F1-score was particularly useful in understanding whether the model was overfitting or underfitting specific classes.
2. **Confusion Matrix Analysis:** We plotted the confusion matrix to visualize the model's performance across classes. This step allowed us to identify common misclassifications and explore relationships between different emotions. By examining the matrix, we fine-tuned the model's training process to address specific confusions.
3. **Averaging Ensemble Predictions:** For ensemble models, we averaged the predictions from each model to obtain the final output. This averaging approach helped to smooth out individual model errors, leading to more reliable predictions. We used weighted averaging based on the validation accuracy of each model to give more importance to better-performing models in the ensemble.
4. **Model Fine-tuning:** Based on the analysis from the confusion matrix and classification report, we further fine-tuned our models by adjusting hyperparameters, rebalancing the dataset using techniques like SMOTE, and experimenting with different combinations in our ensemble models.
5. **Threshold Calibration:** For certain emotions, we adjusted the decision threshold to account for class imbalance. By modifying the threshold, we aimed to enhance sensitivity for minority classes, ensuring the model captures these emotions more effectively.

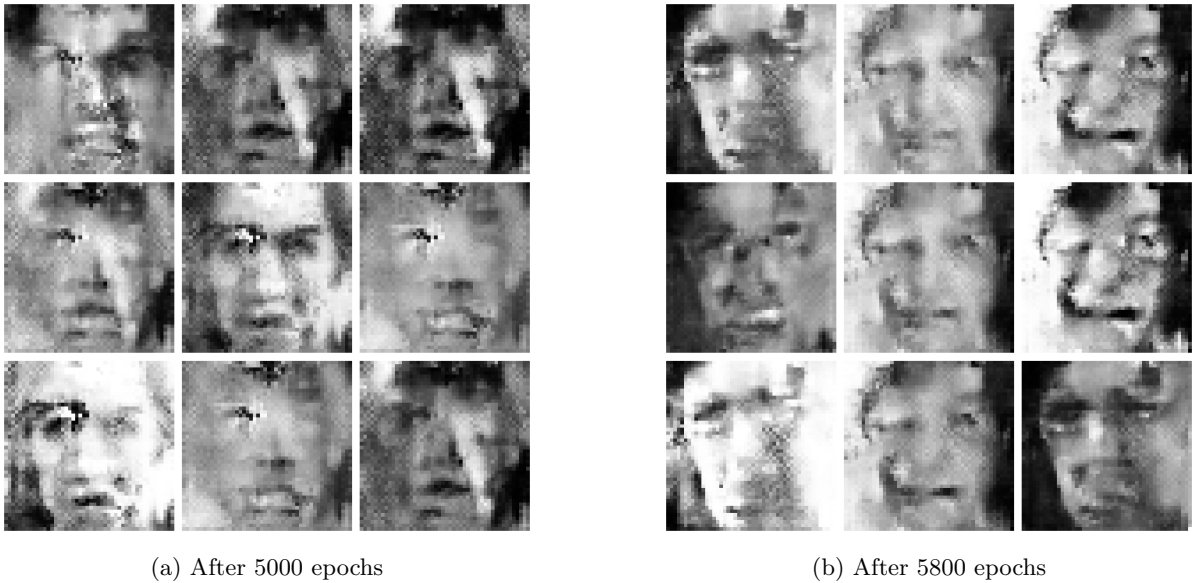


Figure 1: Images generated by GANs during training

2 Innovative Methods

1. **Generative Adversarial Networks (GANs):** We used DCGANs to generate synthetic images for specific classes to address dataset imbalance. Although we primarily focused on the 'Disgust' class, the GAN was not able to produce high-quality face images that could be included in the training set. However, this process helped us understand the limitations and potential of using GANs for data augmentation.
 2. **SMOTE (Synthetic Minority Over-sampling Technique):** To mitigate class imbalance, we utilized SMOTE to generate synthetic samples for the minority classes. By increasing the representation of these classes, the model became more robust in recognizing less frequent emotions.
 3. **Focal Loss:** Switching to focal loss was crucial in training our model on an imbalanced dataset. This loss function emphasizes learning from hard-to-classify examples by down-weighting easy ones, leading to improved performance on minority classes.
 4. **Hyperparameter Optimization:** We used Keras' `Hypermodel` to automate hyperparameter tuning, which helped us identify the optimal set of parameters for model training. This process improved model performance by fine-tuning aspects such as learning rate, batch size, and network architecture.
 5. **Class Weighting:** During training, we assigned weights to each class inversely proportional to the number of samples. This modification in the loss function penalized the misclassification of underrepresented classes, helping the model learn their features more effectively.
 6. **Transformers (ViT, PiT, and SWIN):** We incorporated advanced transformer-based models to improve the recognition of complex emotions:
 - **ViT (Vision Transformer):** Although ViT provided a decent performance, it struggled with spatial inductive biases due to its reliance on patch-based inputs.
 - **SWIN Transformer:** SWIN improved over ViT by introducing hierarchical representations and shifted windows, allowing better handling of varied image sizes and local context. It outperformed ViT.
 - **PiT (Pyramid Vision Transformer):** PiT performed better than SWIN and ViT because of its hierarchical structure and the inclusion of pooling layers, which helped in capturing multi-scale features more effectively.
- Despite their complexities, these transformer models often require more data for effective training, which might have limited their performance compared to some CNNs in our dataset.
7. **Performance Comparison:** Among the traditional CNNs, **EfficientNet** achieved the highest accuracy due to its compound scaling technique, which balances network depth, width, and resolution effectively. However, **ConvNext** outperformed all other models, including transformers, by leveraging a modernized ResNet-based architecture with improved convolutional operations. Its ability to capture fine-grained spatial details gave it an edge over other models in recognizing subtle variations in facial expressions.
 8. **Ensembling with Features:** To further enhance performance, we implemented an ensemble model where features extracted from ConvNext and EfficientNet were combined and passed through a neural network. This combination of complementary feature sets improved the model's overall accuracy.
 9. **Threshold Selection:** We experimented with different thresholds for individual classes to fine-tune the model's sensitivity towards minority emotions. This technique allowed us to better balance the precision-recall trade-off, particularly for classes with fewer samples.